

USB RECOVERY VAULT

Repair and Validation Manual

Windows 11 - NTFS corruption - disk imaging - TestDisk/PhotoRec - key recovery

RECOVERY OUTCOME: SUCCESSFUL

This manual documents the successful recovery of a roughly 32 GB NTFS USB recovery vault that became RAW and unreadable. The procedure preserved the original media, created a sector image, recovered a matching SSH key pair, rebuilt and stress-tested the USB, restored an OpenPGP backup, and removed sensitive temporary artifacts.

Prepared for	Sanjev Rajaram
Date	August 18, 2026
Scope	Windows 11; removable NTFS USB; SSH and OpenPGP recovery material

Privacy note

This manual intentionally omits private-key contents, passwords, serial numbers, email account details, and actual cryptographic fingerprints. Never paste or publish a private key while following these steps.

0

Purpose, scope, and quick recovery map

Use this page as the high-level runbook; detailed steps follow.

Use this procedure when Windows can still detect a USB device but the volume is RAW, inaccessible, or reports that the disk structure is corrupted and unreadable. It is designed for cases where the data matters more than restoring immediate access to the original drive.

The central safety rule

Do not format, repair, or write new files to the original USB until a complete raw image exists. Work on the image whenever possible. A repair attempt can make a recoverable filesystem harder to reconstruct.

Phase	Action	Primary goal
1	Freeze the situation	Stop all writes and preserve the original media.
2	Identify the correct disk	Distinguish the USB from the internal SSD and any SD cards.
3	Diagnose read-only	Confirm partition, boot-sector, and NTFS metadata status.
4	Create a raw image	Capture every readable sector before additional recovery work.
5	Recover from the image	Use TestDisk and PhotoRec without risking the original USB.
6	Validate recovered keys	Prove candidate private/public keys are complete and matching.
7	Rebuild and stress-test	Full format, write test, hashes, reconnect test, and CHKDSK.
8	Restore the vault	Copy verified SSH keys and export the OpenPGP secret backup.
9	Clean up safely	Remove raw images and temporary private-key copies only after verification.

Case summary

- Trigger observed: the USB became unreadable while a new folder was being created from an application Save dialog.
- Windows result: the volume appeared as RAW and reported that the disk structure was corrupted and unreadable.
- TestDisk result: the NTFS boot sector and backup boot sector were healthy and identical; both the MFT and MFT mirror were damaged.
- Recovery result: a complete raw image was created, a valid matching Ed25519 SSH key pair was recovered, and the USB later passed a full rebuild and integrity test.
- Root cause conclusion: the timing was clear, but causation was not proven. The Save dialog operation may have exposed an existing filesystem problem rather than created it.

Tools used

Tool	Use in the recovery
Windows Disk Management	Identify physical disks, confirm RAW status, delete/recreate the volume.
TestDisk	Inspect partition structure, NTFS boot sectors, and MFT condition.
HDD Raw Copy Tool	Create a sector-by-sector image and acquisition SHA-256.
PhotoRec	Carve files from the image when the MFT could not be repaired.
PowerShell / OpenSSH	Search for key markers, restrict permissions, validate fingerprints, and stress-test the USB.
Thunderbird	Export the password-protected OpenPGP secret-key backup.

1

Freeze the situation and protect the original USB

Treat every new write as a potential loss of recoverable metadata.

Immediate actions

1. Stop copying, renaming, deleting, or creating folders on the affected USB.
2. Cancel any application Save dialog that is still pointed at the USB.
3. Do not accept a Windows prompt to format the drive.
4. Do not run CHKDSK /f on the original drive before imaging it.
5. Do not repeatedly unplug and reconnect the USB while applications still have it open.
6. Keep the laptop connected to power and temporarily prevent sleep during long imaging or hash operations.

Why this matters

A damaged NTFS filesystem can still contain intact file contents even when Windows cannot read filenames or folders. Preserving the current sectors gives recovery tools the best chance of reconstructing data.

If Windows is still responsive

- Close File Explorer windows displaying the affected drive.
- Close Thunderbird or any other application that has a Save dialog open to the USB.
- If the interface becomes sluggish, try Ctrl+Alt+Delete before forcing power off. A responsive security screen means Windows is not completely frozen.
- Use Task Manager > Performance to confirm whether the removable disk is actively reading or writing before unplugging it.

Temporary sleep setting

During imaging, a 10 GiB write test, or full-file hashing, set Settings > System > Power & battery > Screen, sleep & hibernate timeouts > Make my device sleep after > Never for Plugged in. Restore the normal security-conscious setting afterward.

Do not confuse screen-off with sleep

The screen may turn off safely, but system sleep suspends PowerShell and USB I/O. Do not close the lid unless the lid action is known not to sleep the laptop.

2

Identify the correct physical disk

Never select a disk by guesswork when capacities are similar.

Use Disk Management first

1. Press Windows+X and open Disk Management.
2. In the lower pane, locate the removable disk that contains the RAW volume.
3. Record its disk number, capacity, drive letter, and label.
4. Also record the internal SSD and any SD card so they cannot be confused with the target.

Device in the successful case	Capacity / status	Action
Disk 0 - internal NVMe SSD	Approximately 477 GB; BitLocker; C:	Never select as source or target for USB recovery.
Disk 1 - removable SD card	Approximately 29.7 GB; exFAT; E:	Leave untouched.
Disk 2 - affected USB	Approximately 28.9 GB; RAW; initially D:	This was the recovery source.

Capacity is not enough by itself

Two removable devices can both be sold as 32 GB yet appear as different usable capacities. Confirm the disk number and volume letter in Disk Management before proceeding.

Confirm again in TestDisk

Run TestDisk as Administrator. Choose Create for the log file. On the disk-selection screen, compare the physical-drive number, capacity, and model with Disk Management. In the successful case, the affected USB was PhysicalDrive2 - USB DISK 2.0 - about 31 GB / 28 GiB.

Safe TestDisk path

5. Select the affected physical disk and choose Proceed.
6. Accept the detected Intel/PC partition table when TestDisk explicitly reports that it detected Intel.
7. Choose Analyse, then Quick Search.
8. Do not choose MBR Code, Delete, Write, or geometry changes on the original USB.
9. If a partition is found, press P only to attempt a read-only file listing.

Expected observation

TestDisk can report Structure: Ok while still being unable to open the NTFS filesystem. That means the partition entry is present, but internal filesystem metadata may be damaged.

3

Diagnose the NTFS failure

Use the result to decide whether to repair, image, or carve files.

What the successful case showed

Component	Result	Meaning
Partition entry	Present; structure reported OK	The partition table still described the main NTFS volume.
Primary NTFS boot sector	OK	Core NTFS geometry and metadata pointers were readable.
Backup boot sector	OK and identical	Rebuilding the boot sector was unnecessary.
MFT	Bad	NTFS could not reliably map filenames, folders, and file extents.
MFT mirror	Bad	The normal metadata fallback was also unusable.
Windows volume state	RAW	Windows could not mount the NTFS filesystem.

How to inspect this on an image

After a raw image exists, open it in TestDisk and choose Intel > Advanced > Boot. The Boot screen reports the primary and backup boot-sector status and offers Repair MFT. In this case, Repair MFT reported that the MFT and MFT mirror were both bad and could not be repaired.

Do not rebuild a healthy boot sector

When both boot sectors are OK and identical, do not choose Rebuild BS. It does not address MFT corruption and can introduce an unnecessary change.

Decision reached

Because the filesystem catalog could not be repaired, the correct next step was to preserve all readable sectors and recover file contents from a raw image rather than continuing to modify the original USB.

Troubleshooting note: Windows repair attempts

In the incident, Windows Error Checking initially found errors and offered Repair. After the repair attempt, the drive became fully inaccessible. This reinforced the rule that irreplaceable data should be imaged before filesystem repair. Once a verified image exists, repair experiments can be performed on copies.

4

Create a sector-by-sector image

The raw image becomes the working copy and the original USB becomes evidence/fallback.

Prepare the destination

- Prefer a local, non-synchronized path such as C:\USB-Recovery\RecoveryVault.img.
- Confirm that the internal drive has at least the full capacity of the USB plus working space.
- Avoid placing the image in OneDrive or another cloud-synchronized folder. The successful case used a OneDrive-backed Documents folder for convenience, but required permanent cloud recycle-bin cleanup later.

Verify the imaging utility

HDD Raw Copy Tool was used because it can read a physical Windows drive into an image file. The downloaded installer was unsigned, so its SHA-256 was calculated and compared with a known release hash before it was allowed to run as Administrator. Do not bypass SmartScreen for a raw-disk utility without verifying the file.

HDD Raw Copy Tool configuration

1. Source: select the affected physical USB. Confirm disk number, capacity, model, and volume letter.
2. Target: select Image file, not another physical drive.
3. Choose a filename ending in .img.
4. Enable SHA-256 hashing.
5. Re-read the summary. It must say Physical USB -> image file on C:.
6. Start the copy and leave the USB connected. Do not allow the laptop to sleep.

Critical source/target check

Selecting the internal SSD or the SD card as a physical target could overwrite it. The destination must be a file path, not a disk.

Successful acquisition criteria

- The copied byte count equals the full physical-device size.
- The tool reports Copy complete.
- No unreadable-sector errors are reported, or any errors are documented if present.
- A SHA-256 acquisition hash is recorded in the report.
- The image file size matches the physical drive size.

Once the image completed, the original USB was safely ejected and stored disconnected. All subsequent recovery work was performed on the .img file.

5

Recover from the raw image

Use TestDisk for structure and PhotoRec for file carving.

Open the image in TestDisk

Example PowerShell launch

```
cd "C:\Path\To\testdisk"  
.\testdisk_win.exe "C:\USB-Recovery\RecoveryVault.img"
```

1. Confirm the top of TestDisk shows the .img path, not PhysicalDrive2.
2. Choose the detected Intel/PC partition table.
3. Choose Advanced > Boot to inspect NTFS boot sectors and MFT status.
4. If Repair MFT fails, quit TestDisk rather than repeatedly attempting repairs.

Carve files with PhotoRec

Example PowerShell launch

```
.\photorec_win.exe "C:\USB-Recovery\RecoveryVault.img"
```

5. Select the NTFS partition inside the image.
6. Leave all file families enabled for the first pass unless storage space is limited.
7. Choose Other for FAT/NTFS/HFS+/ReiserFS.
8. Choose Whole, not Free, because the MFT allocation map is not trustworthy.
9. Choose a separate destination, for example C:\USB-Recovery\Recovered Files.
10. Press C only after the destination shown at the top is correct.

Expected PhotoRec behavior

Original names and folder structure may be lost. PhotoRec creates generic names and recup_dir folders because it identifies files by content signatures rather than relying on the damaged MFT.

If PhotoRec recovers only a small set

A small number of recovered files is not automatically a failure. Small text-based keys can still be present, and candidate files can be identified by key headers even when the filename or extension is generic.

6

Identify and validate recovered SSH keys

Never reveal private-key contents; validate locally by format and fingerprint.

Search recovered files for key markers

Marker-only search (prints paths, not key contents)

```
$recovery = "C:\USB-Recovery\Recovered Files\recup_dir.1"

Get-ChildItem $recovery -File |
  Select-String -Pattern `
    "BEGIN OPENSSSH PRIVATE KEY", `
    "BEGIN PRIVATE KEY", `
    "BEGIN PGP", `
    "ssh-ed25519", `
    "ssh-rsa" `
  -SimpleMatch -List |
  Select-Object -ExpandProperty Path
```

Test a recovered private-key candidate

OpenSSH on Windows rejects private-key files with permissive ACLs. Test a copy in a restricted local folder rather than changing the PhotoRec original.

Validate the private key without displaying it

```
$test = "$env:USERPROFILE\SSH-Key-Recovery-Test"
New-Item -ItemType Directory -Force $test | Out-Null
Copy-Item "C:\Path\candidate.txt" "$test\recovered_private_key"

icacls "$test\recovered_private_key" /inheritance:r
icacls "$test\recovered_private_key" /grant:r "${env:USERNAME}:F"

ssh-keygen -l -f "$test\recovered_private_key"
```

A valid result reports the key size, SHA-256 fingerprint, optional comment, and key type such as ED25519. An error stating that the file is not a key file, after ACL warnings are gone, means the candidate is not a usable OpenSSH key.

Confirm the matching public key

Compare fingerprints

```
ssh-keygen -l -f "C:\Path\candidate_public.txt"
ssh-keygen -l -f "$env:USERPROFILE\.ssh\known_recovery_key.pub"
```

The recovered private key, recovered public key, and known public-key copy must report the same fingerprint. Matching fingerprints prove that the files belong to the same key pair. Comments may differ without changing the key.

Do not use screenshots containing private-key text

Fingerprints and public-key comments are safe to record. The private-key block itself is not. Do not paste the private key into email, chat, issue trackers, or Git repositories.

7

Rebuild and stress-test the USB

Reuse only after a full format and a reproducible write-retain-read test.

Recreate the volume

1. Reconnect the original USB only after the raw image and important files are verified elsewhere.
2. Open Disk Management and positively identify the USB again.
3. Right-click the RAW volume and choose Delete Volume.
4. Right-click the resulting unallocated space and choose New Simple Volume.
5. Use the maximum size, NTFS, Default allocation unit size, and a clear label such as Recovery Vault.
6. For this trust test, perform a full format rather than a quick format.

Initial filesystem check

Replace F: with the current Recovery Vault drive letter

```
chkdsk F: /f
```

On removable NTFS media, chkdsk /scan may fail because snapshot scanning is unsupported. The offline /f check is appropriate when no files are open on the USB. The desired result is no filesystem problems and 0 KB in bad sectors.

Write a 10 GiB random-data test with progress

Before running

Keep the laptop plugged in, set sleep to Never while plugged in, close applications using the USB, and confirm the drive letter.

PowerShell stress-test script

```
$path = "F:\USB-STRESS-TEST.bin"
$sizeGiB = 10
$blocks = $sizeGiB * 1024
$buffer = New-Object byte[] (1MB)
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$fs = [System.IO.File]::Open(
    $path,
    [System.IO.FileMode]::Create,
    [System.IO.FileAccess]::Write,
    [System.IO.FileShare]::None
)

try {
    for ($i = 0; $i -lt $blocks; $i++) {
        $rng.GetBytes($buffer)
        $fs.Write($buffer, 0, $buffer.Length)
        if (($i % 64) -eq 0) {
            Write-Progress -Activity "USB stress test" `
                -Status "$($i + 1) of $blocks MiB" `
                -PercentComplete ((($i + 1) / $blocks) * 100)
        }
    }
    $fs.Flush($true)
}
finally {
    $fs.Dispose()
    $rng.Dispose()
    Write-Progress -Activity "USB stress test" -Completed
}
```

7B**Verify retention across a physical reconnect***A matching hash proves the test file remained byte-for-byte identical.***Create Hash #1****Read and record the full-file SHA-256**

```
$hash1 = (Get-FileHash "F:\USB-STRESS-TEST.bin" -Algorithm SHA256).Hash
$hash1 | Set-Content "$env:USERPROFILE\Desktop\USB-STRESS-HASH.txt"
$hash1
```

Physically reconnect

1. Close File Explorer windows displaying the USB.
2. Use Safely Remove Hardware or the File Explorer Eject button.
3. Wait for Windows to say it is safe, unplug the USB, wait about 10 seconds, and reconnect it.
4. Confirm the drive label and note whether the drive letter changed.

Create Hash #2 and compare**The comparison must return True**

```
$expected = Get-Content "$env:USERPROFILE\Desktop\USB-STRESS-HASH.txt"
$actual = (Get-FileHash "F:\USB-STRESS-TEST.bin" -Algorithm SHA256).Hash
$expected -eq $actual
$actual
```

Final filesystem check and cleanup**Run CHKDSK before deleting the test file**

```
chkdsk F: /f
Remove-Item "F:\USB-STRESS-TEST.bin"
```

USB acceptance criteria

Accept the USB for reuse only if the full format completes, CHKDSK is clean, the 10 GiB write completes without error, Hash #1 and Hash #2 match exactly after reconnect, and the final CHKDSK remains clean.

Passing this test is strong evidence of current reliability, not a permanent guarantee. Any flash device can fail later, so critical recovery material should not exist on only one physical medium.

8

Restore the Recovery Vault

Create the folder structure first, then copy and verify each recovery artifact.

Create folders in ordinary File Explorer or PowerShell

Pre-create the destination folders

```
New-Item -ItemType Directory `
-Path "F:\Recovery-Key-Vault\Windsor-Onion-Server" -Force

New-Item -ItemType Directory `
-Path "F:\Recovery-Key-Vault\Gmail-OpenPGP" -Force
```

Workflow lesson from the incident

Create folders before opening an application export dialog. In the successful rebuild, the Gmail-OpenPGP folder was created first; Thunderbird only navigated to it and saved the file.

Restore and verify the SSH recovery pair

Copy and fingerprint the USB copies

```
Copy-Item "$env:USERPROFILE\Onion-Recovery-Restored\onionserver_recovery" `
"F:\Recovery-Key-Vault\Windsor-Onion-Server\onionserver_recovery"

Copy-Item "$env:USERPROFILE\Onion-Recovery-Restored\onionserver_recovery.pub" `
"F:\Recovery-Key-Vault\Windsor-Onion-Server\onionserver_recovery.pub"

ssh-keygen -l -f `
"F:\Recovery-Key-Vault\Windsor-Onion-Server\onionserver_recovery"
ssh-keygen -l -f `
"F:\Recovery-Key-Vault\Windsor-Onion-Server\onionserver_recovery.pub"
```

Both USB files must report the same fingerprint that was previously verified on the laptop. Repeat this check after physically ejecting and reconnecting the USB.

Export the Thunderbird OpenPGP secret key

1. Open Thunderbird > Account Settings > End-To-End Encryption > OpenPGP Key Manager.
2. Select the personal OpenPGP key and choose Back Up Secret Key(s) to File or Export Secret Key.
3. Navigate to F:\Recovery-Key-Vault\Gmail-OpenPGP. Do not create folders from the Save dialog.
4. Save the ASCII-armored .asc file and protect the export with a strong, unique backup password.
5. Do not store that password in the same folder as the secret-key backup.

Validate the OpenPGP backup without displaying it

Expected: True, followed by a SHA-256 hash

```
$pgp = Get-ChildItem "F:\Recovery-Key-Vault\Gmail-OpenPGP\*.asc" |
Select-Object -First 1

Select-String -Path $pgp.FullName `
-Pattern "BEGIN PGP PRIVATE KEY BLOCK" `
-SimpleMatch -Quiet

Get-FileHash $pgp.FullName -Algorithm SHA256
```

The marker check confirms that the file is an armored OpenPGP private-key backup. The SHA-256 is a file-integrity checksum, not the OpenPGP key fingerprint. Record it separately if desired.

9

Secure cleanup and closure

Delete temporary private-key copies only after the vault survives reconnect verification.

Final verification before deleting anything

- [] SSH recovery private key validates from the USB.
- [] SSH recovery public key validates from the USB.
- [] Private and public SSH fingerprints match.
- [] OpenPGP .asc marker check returns True.
- [] OpenPGP backup SHA-256 is recorded.
- [] USB is safely ejected, reconnected, and the checks still match.

Remove temporary recovery locations

The successful case used three temporary locations: a key-test folder, a clean staging folder, and a large USB Recovery folder containing the raw image and PhotoRec output. Delete these only after the USB is independently verified.

Each Test-Path should return False

```
Remove-Item "$env:USERPROFILE\SSH-Key-Recovery-Test" -Recurse -Force
Remove-Item "$env:USERPROFILE\Onion-Recovery-Restored" -Recurse -Force
Remove-Item "$env:USERPROFILE\OneDrive\Documents\USB Recovery" -Recurse -Force

Test-Path "$env:USERPROFILE\SSH-Key-Recovery-Test"
Test-Path "$env:USERPROFILE\Onion-Recovery-Restored"
Test-Path "$env:USERPROFILE\OneDrive\Documents\USB Recovery"
```

OneDrive cleanup is separate

Deleting a synchronized folder locally can leave a cloud-recoverable copy in the OneDrive Recycle Bin. Permanently delete only the USB Recovery folder from the OneDrive Recycle Bin; do not empty unrelated deleted items unless intended.

Restore normal security settings

- Restore the laptop sleep and screen-lock timeouts that were temporarily set to Never.
- Safely eject and physically store the Recovery Vault USB.
- Keep the OpenPGP backup password in a separate secure location.
- Do not delete the normal .ssh folder or the active primary SSH key on the laptop.

Storage recommendation

A single USB is a recovery copy, not a complete backup strategy. For critical keys, maintain a second encrypted offline copy on different media and periodically validate both copies. Keep the media in separate physical locations when practical.

10

Lessons learned and troubleshooting

Common symptoms, correct interpretation, and safer future habits.

What the evidence did and did not prove

Observation	Reasonable conclusion
The failure occurred while creating a folder from Thunderbird's Save dialog.	The operation coincided with the failure, but it did not prove Thunderbird caused NTFS corruption.
The primary and backup boot sectors were healthy.	The failure was not an NTFS boot-sector problem.
Both MFT copies were damaged.	Windows lost the filesystem catalog needed to mount the volume.
The raw image copied without reported read errors.	Most or all physical sectors remained readable at acquisition time.
A 10 GiB random file survived reconnect with an identical SHA-256.	The rebuilt USB correctly wrote, retained, and reread that test data.
Final CHKDSK remained clean.	No filesystem inconsistency was detected after the stress test.

PowerShell continuation prompt

When PowerShell displays `>>`, it is waiting for more input. After entering a multiline block, press Enter on a blank continuation line to execute it. Press Ctrl+C to cancel an unfinished block. A blinking cursor with no prompt often means the command is actively running.

How to confirm a long operation is active

- The USB activity light blinks during I/O.
- Task Manager > Performance > the removable disk shows nonzero read/write speed and high active time.
- The normal PowerShell prompt returns only after the operation completes.

If the laptop appears frozen

1. Wait briefly if the USB is actively reading or writing.
2. Try Ctrl+Alt+Delete. If the security screen appears, Windows is still responsive.
3. Open Task Manager and check memory and disk activity.
4. Avoid forced power-off unless Ctrl+Alt+Delete and Task Manager are both unresponsive for several minutes.
5. After any forced shutdown or unsafe removal, run CHKDSK on the USB before trusting it again.

Minimum future routine

Every few months: connect the Recovery Vault, verify the SSH fingerprint, hash the OpenPGP backup, safely eject, reconnect, and repeat the small-file checks. A full 10 GiB stress test is not needed every time.

11

Quick command reference

Use only after confirming drive letters and paths.

Replace F: if necessary

Windows can assign a different drive letter after reconnecting. Confirm the Recovery Vault label in File Explorer or Disk Management before running any command.

Filesystem and integrity

```
chkdsk F: /f
Get-FileHash "F:\USB-STRESS-TEST.bin" -Algorithm SHA256
Remove-Item "F:\USB-STRESS-TEST.bin"
```

SSH key validation

```
ssh-keygen -l -f "F:\Recovery-Key-Vault\Windsor-Onion-Server\onionserver_recovery"
ssh-keygen -l -f "F:\Recovery-Key-Vault\Windsor-Onion-Server\onionserver_recovery.pub"
```

OpenPGP backup validation

```
$pgp = Get-ChildItem "F:\Recovery-Key-Vault\Gmail-OpenPGP\*.asc" |
    Select-Object -First 1

Select-String -Path $pgp.FullName `
    -Pattern "BEGIN PGP PRIVATE KEY BLOCK" -SimpleMatch -Quiet
Get-FileHash $pgp.FullName -Algorithm SHA256
```

Folder structure check

```
Get-ChildItem "F:\Recovery-Key-Vault" -Recurse
```

Final manual checklist

- [] The original USB was imaged before destructive repair or formatting.
- [] The correct physical disk was identified in Disk Management and the recovery tool.
- [] Recovery work was performed on the image, not the original media.
- [] Recovered private keys were never displayed or shared.
- [] Matching key fingerprints were verified.
- [] The USB passed full format, CHKDSK, random-data write, reconnect, and hash comparison.
- [] The Recovery Vault folders were created before the Thunderbird export.
- [] The OpenPGP .asc backup was password-protected and validated.
- [] Temporary raw images and recovered private-key copies were removed.
- [] The OneDrive Recycle Bin was cleared of the sensitive recovery folder.
- [] Normal sleep and lock settings were restored.
- [] The Recovery Vault was safely ejected and securely stored.

End state

The repaired USB is a verified recovery vault containing a matching SSH recovery key pair and a password-protected OpenPGP secret-key backup. The laptop retains the working Thunderbird key and normal SSH configuration, while temporary forensic copies have been removed.